SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR HOTEL RESERVATION AND GUEST SERVICES MANAGEMENT SYSTEM

Program	B. Tech – Computer Science and Engineering
Section	AI-ML Rhinos
Faculty	Prof. Sahithi
Date of Submission	15th October 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102082	**Varshith Godari**	**. 7780325152**	varshith.godari_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Prof. Sahithi, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **"Design and Implementation of a Database Management System for Hotel Reservation and Guest Services Management System"**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date : **15th October 2026**

Place : **Woxsen University**

Name: Varshith godari

Roll Number: 25WU0102082

Program: B. Tech – Computer Science and Engineering

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9-12
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	10
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	11
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	12
2. LOGICAL DATABASE DESIGN	13-22
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	13-15
2.3 Functional Dependencies	15-16
2.4 Normalization up to Third Normal Form (3NF)	16-17
2.5 Data Dictionary	17-21
2.6 Implementation of Logical Design: DDL and Sample Data	21-22
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	23-29
3.1 Application Overview	23
3.2 Development Environment and Database Connectivity	23
3.3 Application Modules and Database Mapping	23-24
3.4 CRUD Operations	24
3.5 Business Rules, Validation and Error Handling	24-25
3.6 Reports and Database Operations	25-26

3.7 Application Screens	26-28
3.8 Testing and Results	28-29
4. CONCLUSION	30
4.1 Limitations and Future Enhancements	30
5. REFERENCES	31
6. APPENDICES	32

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	11
Table 4	Entity and Relationship Summary	12
Table 5	Relational Schema	13
Table 6	Keys and Integrity Constraints	13-15
Table 7	Normalization Status up to 3NF	17
Table 8	Data Dictionary	17-21
Table 9	Database Implementation Environment	21
Table 10	Application Development Environment	23
Table 11	Application Module-to-Database Mapping	23-24
Table 12	CRUD Operations	24
Table 13	Business Rules, Validation and Error Handling	25
Table 14	Reports and Database Operations	25-26
Table 15	Test Cases	28-29

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Representative DDL Implementation in SQLite	22
Figure 3	Hotel Reservation and Guest Services Management Dashboard	26
Figure 4	Guest Management and Guest Record Display	26
Figure 5	Room Reservation and Reservation Details	27
Figure 6	Housekeeping Task Details	27
Figure 7	Payment Transaction and Payment Record	28

LIST OF ACRONYMS

Acronym	Expansion
DBMS	Database Management System
DDL	Data Definition Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
PK	Primary Key
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
3NF	Third Normal Form
PBL	Project-Based Learning

ABSTRACT

Hotel management involves maintaining accurate information about guests, rooms, reservations, stays, services, housekeeping activities, invoices, and payments. When these activities are managed through disconnected or manual processes, hotels may face problems such as double booking, difficulty tracking guest information, uncoordinated service requests, and inconsistencies in billing records. The objective of this project is to design and implement a database management system for hotel reservation and guest services management that centralizes these operations in a structured relational database. The proposed system is developed using an Entity-Relationship model and converted into a normalized relational design up to Third Normal Form (3NF). SQLite is used as the database, while a Python Flask application provides the application interface. The system supports guest and room management, reservations, check-in and check-out, service requests, housekeeping tasks, service charges, invoices, payments, and database-based reports. Validation rules are implemented to maintain data integrity, including prevention of overlapping reservations and validation of transaction-related information. The completed prototype demonstrates how DBMS concepts such as relational modelling, normalization, constraints, CRUD operations, transactions, and SQL queries can be applied to a practical hotel management application.

KEYWORDS

Hotel Reservation, Relational Database, ER Diagram, Normalization, SQLite, Flask

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1. Problem Statement

Hotels need to manage several interconnected activities such as guest registration, room allocation, reservations, stays, service requests, housekeeping, billing, and payments. When these activities are maintained manually or through disconnected records, it becomes difficult to maintain consistent and accurate information. Room availability may not be updated properly, which can result in overlapping reservations. Guest details, service requests, and charges may also become difficult to track throughout a stay.

The proposed Hotel Reservation and Guest Services Management System addresses these problems by providing a centralized database application for managing hotel operations. The system stores related information using structured relational tables and maintains relationships between guests, rooms, reservations, stays, services, housekeeping tasks, invoices, and payments. Database constraints and application-level validation are used to improve data consistency and enforce important business rules.

2. Objectives

The project has the following objectives:

- Design a relational database for managing hotel reservations and guest services.
- Develop a Python-based application connected to the hotel database.
- Implement modules for guests, rooms, reservations, stays, services, housekeeping, billing, and payments.
- Validate key business rules such as room availability, reservation dates, and payment details, while generating useful reports and outputs to support hotel operations.

3. Scope and Assumptions

The topics in scope, out of scope and in assumptions are as follows:

Table 1. Scope and Assumptions

Item	Description
In Scope	Guest management, room and room-type management, rate plans, reservations, stays, service requests, housekeeping tasks, service charges, invoices, payments, and reports.
Out of Scope / Exclusions	Online payment gateway integration, real-time multi-hotel management, external booking-platform integration, and deployment as a production cloud system.
Assumptions	Hotel staff enter valid guest and reservation information. The system is used for a single hotel. Sample records are used for prototype testing.

4. Proposed Solution Strategy

The proposed solution to handle this hotel management and guest services problem is:

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Reception staff, housekeeping staff, billing staff, managers/administrators
Major Application Modules	Guest management, room management, reservations, stays, services, housekeeping, invoices, payments and reports
Proposed Workflow	Guest Registration → Room Availability → Reservation → Check-in → Stay → Services/Housekeeping → Billing → Payment → Check-out → Reports
Important Business Rules	Prevent overlapping reservations, validate stay dates, maintain room capacity, prevent invalid charges and maintain payment/invoice relationships
Expected Reports / Outputs	Room/reservation information, guest information, service and billing information, payment records and database reports

The proposed system uses a structured relational database to manage these operations efficiently [1].

5. Functional Requirements

The functional requirements and their priorities are:

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall allow hotel staff to add and manage guest information.	High
FR-02	The system shall maintain room, room type and rate plan information.	High
FR-03	The system shall create and manage guest reservations while validating room availability.	High
FR-04	The system shall record stays, service requests, housekeeping tasks and service charges.	High
FR-05	The system shall manage invoices and payments and provide relevant reports.	High

6. Entity-Relationship Diagram (ERD)

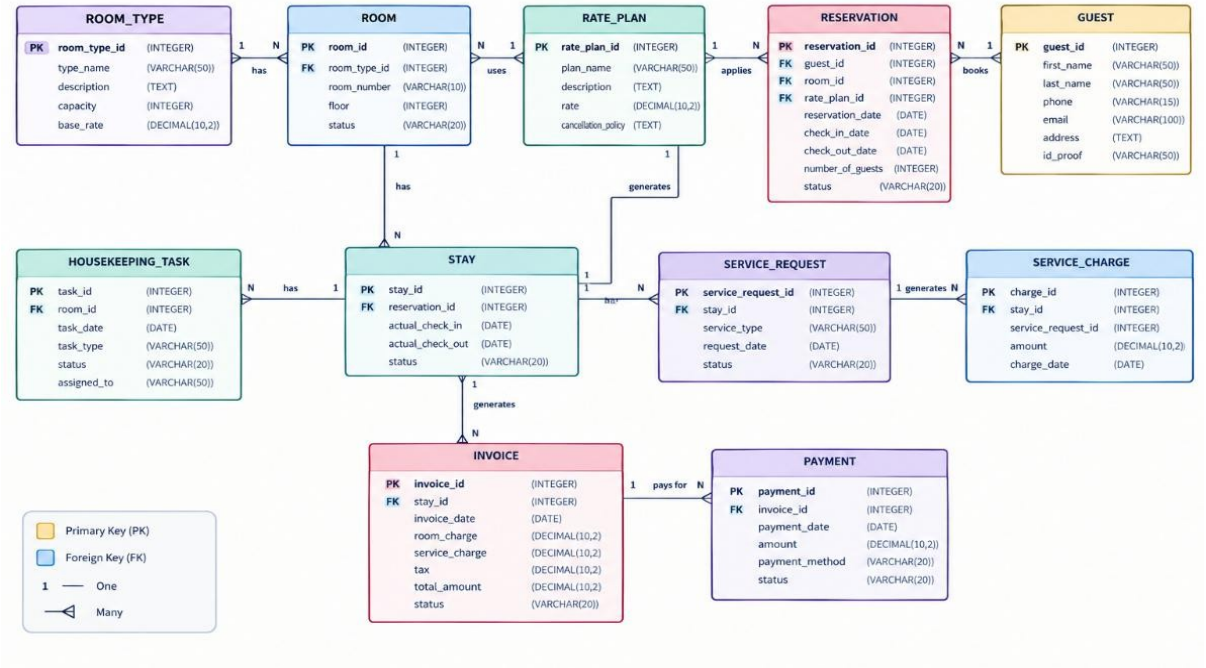


Figure 1. Entity-Relationship Diagram of the Proposed System

The Entity-Relationship model is used to represent entities, attributes, and relationships before converting the design into relational tables [1], [2].

7. Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
ROOM_TYPE → ROOM	room_type_id, room_id; 1:N	Defines room categories and the individual rooms belonging to each category.
GUEST → RESERVATION ← ROOM_PLAN	guest_id, reservation_id, room_id, rate_plan_id; 1:N	Connects guests with room reservations and applicable rate plans. Overlapping reservations for the same room are not allowed.
RESERVATION → STAY → SERVICE_CHARGE	stay_id; 1:0..1, 1:N, 1:N	Records the actual guest stay and services requested or charged during the stay.
ROOM → HOUSEKEEPING_TASK	room_id, task_id; 1:N	Tracks housekeeping activities performed for hotel rooms.
STAY → INVOICE → PAYMENT	invoice_id, payment_id; 1:1, 1:N	Manages billing and payment transactions.

CHAPTER 2

LOGICAL DATABASE DESIGN

1. Relational Schema

The approved ER model is converted into the following relational schema:

Table 5. Relational Schema

S. No.	Table Name	Purpose
1	ROOM_TYPE	Stores room categories, capacity and base rate.
2	ROOM	Stores individual room details and status.
3	RATE_PLAN	Stores room pricing plans and cancellation policies.
4	GUEST	Stores guest details.
5	RESERVATION	Stores room reservation details.
6	STAY	Stores actual check-in and check-out information.
7	SERVICE_REQUEST	Stores guest service requests.
8	HOUSEKEEPING_TASK	Stores housekeeping activities.
9	SERVICE_CHARGE	Stores charges for services provided during a stay.
10	INVOICE	Stores billing information for a stay.
11	PAYMENT	Stores payments made against invoices.

The ER model is converted into a relational schema in which entities are represented as relations with appropriate keys and attributes [1], [2].

2. Keys and Integrity Constraints

The keys and integrity constraints used in the project implementation are:

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
ROOM_TYPE	Primary Key	`room_type_id`	Uniquely identifies each room type.
	UNIQUE, NOT NULL, CHECK	`type_name, capacity, base_rate`	Prevents duplicate room types and invalid capacity/rates.
ROOM	Primary Key	`room_id`	Uniquely identifies each room.
	Foreign Key, UNIQUE	`room_type_id, room_number`	Links each room to a valid room type and prevents duplicate room numbers.

	DEFAULT, CHECK	status, floor	Maintains valid room status and floor values.
RATE_PLAN	Primary Key	rate_plan_id	Uniquely know rate plan.
	UNIQUE, CHECK	plan_name, rate	Prevents duplicate plans and negative rates.
GUEST	Primary Key	guest_id	Uniquely identifies each guest.
	UNIQUE, NOT NULL	email, first_name, last_name, phone	Maintains unique and required guest information.
RESERVATION	Primary Key	reservation_id	Uniquely identifies each reservation.
	Foreign Keys	guest_id, room_id, rate_plan_id	Ensures valid guest, room and rate-plan references.
	CHECK, DEFAULT	Dates, number_of_guests, status	Ensures valid reservation dates, guest count and status.
STAY	Primary Key, UNIQUE	stay_id, reservation_id	Uniquely identifies a stay and allows at most one stay per reservation.
	Foreign Key, DEFAULT, CHECK	reservation_id, status	Maintains valid reservation and stay status.
SERVICE_REQUEST	Primary Key	service_request_id	Uniquely identifies each service request.
	Foreign Key, DEFAULT, CHECK	stay_id, request_date, status	Associates requests with valid stays and maintains valid status.
HOUSEKEEPING_TASK	Primary Key	task_id	Uniquely identifies each housekeeping task.
	Foreign Key, DEFAULT, CHECK	room_id, status	Associates tasks with rooms and maintains valid task status.
SERVICE_CHARGE	Primary Key	charge_id	Uniquely identifies each service charge.
	Foreign Keys, CHECK	stay_id, service_request_id, amount	Links charges to stays/ services and prevents negative amounts.
INVOICE	Primary Key, UNIQUE	invoice_id, stay_id	Uniquely identifies invoices and allows one invoice per stay.

INVOICE	Foreign Key, DEFAULT, CHECK	`stay_id, amount fields, status`	Maintains valid billing values and invoice status.
PAYMENT	Primary Key	`payment_id`	Uniquely identifies each payment.
	Foreign Key, CHECK	`invoice_id, amount, payment_method, status`	Links payments to invoices and restricts

			invalid payment values/ methods.

Primary keys uniquely identify records, while foreign keys maintain relationships between related tables and support referential integrity [1], [4].

3. Functional Dependencies

The major functional dependencies identified in the database design are:

1. ROOM_TYPE

   ```
   room_type_id → type_name, description, capacity,
   base_rate
   ```

2. ROOM

   ```
   room_id → room_number, room_type_id, floor, status
   ```

3. RATE_PLAN

   ```
   rate_plan_id → plan_name, description, rate,
   cancellation_policy
   ```

4. GUEST

   ```
   guest_id → first_name, last_name, phone, email, address,
   id_proof
   ```

5. RESERVATION

   ```
   reservation_id → guest_id, room_id, rate_plan_id,
   reservation_date, check_in_date, check_out_date,
   number_of_guests, status
   ```

6. STAY

   ```
   stay_id → reservation_id, actual_check_in,
   ```

```
actual_check_out, status
```

7. SERVICE_REQUEST

   ```
   service_request_id → stay_id, service_type, request_date,
   status
   ```

8. HOUSEKEEPING_TASK

   ```
   task_id → room_id, task_date, task_type, status,
   assigned_to
   ```

9. SERVICE_CHARGE

   ```
   charge_id → stay_id, service_request_id, amount,
   charge_date
   ```

10. INVOICE

    ```
    invoice_id → stay_id, invoice_date, room_charge,
    service_charge, tax, total_amount, status
    ```

11. PAYMENT

    ```
    payment_id → invoice_id, payment_date, amount,
    payment_method, status
    ```

These dependencies show that the non-key attributes of each relation are determined by its corresponding primary key, supporting the normalized relational design.

4. Normalization

First Normal Form (1NF):

All tables contain atomic values, and repeating groups or multi-valued attributes are avoided.

Second Normal Form (2NF):

All relations use single-attribute primary keys. Therefore, every non-key attribute is fully dependent on the complete primary key, with no partial dependencies.

Third Normal Form (3NF):

Non-key attributes depend only on the primary key and not on other non-key attributes. Related information is separated into appropriate relations such as ROOM_TYPE, ROOM, GUEST, RESERVATION, INVOICE and PAYMENT.

Therefore, the database design is maintained in Third Normal Form (3NF).

Normalization reduces redundancy and helps prevent insertion, deletion, and update anomalies in relational databases [1], [2].

Table 7. Normalization Status up to 3NF

Relation	1NF	2NF	3NF	Remarks
ROOM_TYPE	✓	✓	✓	Atomic attributes; no partial or transitive dependency.
ROOM	✓	✓	✓	Room attributes depend on `room_id`.
RATE_PLAN	✓	✓	✓	Rate-plan attributes depend on `rate_plan_id`.
GUEST	✓	✓	✓	Guest attributes depend on `guest_id`.
RESERVATION	✓	✓	✓	Reservation attributes depend on `reservation_id`.
STAY	✓	✓	✓	Stay attributes depend on `stay_id`.
SERVICE_REQUEST	✓	✓	✓	Service-request attributes depend on `service_request_id`.
HOUSEKEEPING_TASK	✓	✓	✓	Task attributes depend on `task_id`.
SERVICE_CHARGE	✓	✓	✓	Charge attributes depend on `charge_id`.
INVOICE	✓	✓	✓	Invoice attributes depend on `invoice_id`.
PAYMENT	✓	✓	✓	Payment attributes depend on `payment_id`.

5. Data Dictionary

The database has following 11 tables:

Table 8.1. ROOM_TYPE

Attribute	Data Type	Key / Constraint	Description

room_type_id	INTEGER	PK, AUTOINCREMENT	Unique room type identifier.
type_name	TEXT	NOT NULL, UNIQUE	Name of the room type.
description	TEXT	—	Description of the room type.

capacity	INTEGER	NOT NULL, CHECK > 0	Maximum number of guests.
base_rate	REAL	NOT NULL, CHECK ≥ 0	Base rate of the room type.

Table 8.2. ROOM

Attribute	Data Type	Key / Constraint	Description
room_id	INTEGER	PK, AUTOINCREMENT	Unique room identifier.
room_number	TEXT	NOT NULL, UNIQUE	Hotel room number.
room_type_id	INTEGER	FK, NOT NULL	References ROOM_TYPE.
floor	INTEGER	NOT NULL, CHECK ≥ 0	Floor on which the room is located.
status	TEXT	NOT NULL, DEFAULT	Current room status.

Table 8.3. RATE_PLAN

Attribute	Data Type	Key / Constraint	Description
rate_plan_id	INTEGER	PK, AUTOINCREMENT	Unique rate plan identifier.
plan_name	TEXT	NOT NULL, UNIQUE	Name of the rate plan.
description	TEXT	—	Description of the plan.
rate	REAL	NOT NULL, CHECK ≥ 0	Rate associated with the plan.
cancellation_policy	TEXT	—	Cancellation terms.

Table 8.4. GUEST

Attribute	Data Type	Key / Constraint	Description
guest_id	INTEGER	PK, AUTOINCREMENT	Unique guest identifier.
first_name	TEXT	NOT NULL	Guest first name.
last_name	TEXT	NOT NULL	Guest last name.
phone	TEXT	NOT NULL	Guest contact number.
email	TEXT	NOT NULL, UNIQUE	Guest email address.

address	TEXT	—	Guest address.
id_proof	TEXT	—	Guest identification information.

Table 8.5. RESERVATION

Attribute	Data Type	Key / Constraint	Description
reservation_id	INTEGER	PK, AUTOINCREMENT	Unique reservation identifier.
guest_id	INTEGER	FK, NOT NULL	References GUEST.

room_id	INTEGER	FK, NOT NULL	References ROOM.
rate_plan_id	INTEGER	FK, NOT NULL	References RATE_PLAN.
reservation_date	TEXT	NOT NULL, DEFAULT	Date reservation was created.
check_in_date	TEXT	NOT NULL	Planned check-in date.
check_out_date	TEXT	NOT NULL	Planned check-out date.
number_of_guests	INTEGER	NOT NULL, CHECK > 0	Number of guests.
status	TEXT	NOT NULL, DEFAULT	Reservation status.

Table 8.6. STAY

Attribute	Data Type	Key / Constraint	Description
stay_id	INTEGER	PK, AUTOINCREMENT	Unique stay identifier.
reservation_id	INTEGER	FK, UNIQUE, NOT NULL	References RESERVATION.
actual_check_in	TEXT	—	Actual check-in date/time.
actual_check_out	TEXT	—	Actual check-out date/ time.
status	TEXT	NOT NULL, DEFAULT	Current stay status.

Table 8.7. SERVICE_REQUEST

Attribute	Data Type	Key / Constraint	Description
service_request_id	INTEGER	PK, AUTOINCREMENT	Unique service request identifier.
stay_id	INTEGER	FK, NOT NULL	References STAY.
service_type	TEXT	NOT NULL	Type of service requested.
request_date	TEXT	NOT NULL, DEFAULT	Date/time of request.

status	TEXT	NOT NULL, DEFAULT	Current request status.

Table 8.8. HOUSEKEEPING_TASK

Attribute	Data Type	Key / Constraint	Description
task_id	INTEGER	PK, AUTOINCREMENT	Unique housekeeping task identifier.
room_id	INTEGER	FK, NOT NULL	References ROOM.
task_date	TEXT	NOT NULL	Date of housekeeping task.
task_type	TEXT	NOT NULL	Type of housekeeping activity.
status	TEXT	NOT NULL, DEFAULT	Task status.
assigned_to	TEXT	—	Person assigned to the task.

Table 8.9. SERVICE_CHARGE

Attribute	Data Type	Key / Constraint	Description
charge_id	INTEGER	PK, AUTOINCREMENT	Unique charge identifier.
stay_id	INTEGER	FK, NOT NULL	References STAY.
service_request_id	INTEGER	FK	References SERVICE_REQUEST.
amount	REAL	NOT NULL, CHECK ≥ 0	Amount charged for the service.
charge_date	TEXT	NOT NULL, DEFAULT	Date of the charge.

Table 8.10. INVOICE

Attribute	Data Type	Key / Constraint	Description
invoice_id	INTEGER	PK, AUTOINCREMENT	Unique invoice identifier.
stay_id	INTEGER	FK, UNIQUE, NOT NULL	References STAY.
invoice_date	TEXT	NOT NULL, DEFAULT	Invoice creation date.
room_charge	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0	Room-related charge.
service_charge	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0	Total service charge.
tax	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0	Tax amount.
total_amount	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0	Total invoice amount.
status	TEXT	NOT NULL, DEFAULT	Invoice payment state

Table 8.11. PAYMENT

Attribute	Data Type	Key / Constraint	Description
payment_id	INTEGER	PK, AUTOINCREMENT	Unique payment identifier.
invoice_id	INTEGER	FK, NOT NULL	References INVOICE.
payment_date	TEXT	NOT NULL, DEFAULT	Date of payment.
amount	REAL	NOT NULL, CHECK > 0	Payment amount.
payment_method	TEXT	NOT NULL, CHECK	Cash, Card, UPI or Bank Transfer.
status	TEXT	NOT NULL, DEFAULT,	Successful or Failed

		CHECK	payment.

6. Implementation of Logical Design: DDL and Sample Data

The logical database design was implemented using **SQLite** as the relational database management system. The database contains 11 tables. The tables were created using SQL Data Definition Language (DDL) with appropriate primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. These constraints help maintain data integrity and enforce important business rules of the hotel reservation system. SQLite supports table constraints such as primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK constraints [3], [4].

Table 9. Database Implementation Environment

Item	Details
DBMS	SQLite
Database File	`hotel.db`
Number of Tables	11
Application Language	Python
Database Connectivity	Python SQLite3
Sample Data	Room types, rooms, rate plans, guests, reservations, stays, service requests, housekeeping tasks, service charges, invoices and payments

The project uses SQLite and enables foreign-key enforcement through `PRAGMA foreign_keys = ON`.

```
CREATE TABLE IF NOT EXISTS room_type (
    room_type_id INTEGER PRIMARY KEY AUTOINCREMENT,
    type_name TEXT NOT NULL UNIQUE,
    description TEXT,
    capacity INTEGER NOT NULL CHECK (capacity > 0),
    base_rate REAL NOT NULL CHECK (base_rate >= 0)
);
```

Hotel_DBMS_Final_Project > database > schema.sql

```sql
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS room_type (
    room_type_id INTEGER PRIMARY KEY AUTOINCREMENT,
    type_name TEXT NOT NULL UNIQUE,
    description TEXT,
    capacity INTEGER NOT NULL CHECK (capacity > 0),
    base_rate REAL NOT NULL CHECK (base_rate >= 0)
);

CREATE TABLE IF NOT EXISTS room (
    room_id INTEGER PRIMARY KEY AUTOINCREMENT,
    room_number TEXT NOT NULL UNIQUE,
    room_type_id INTEGER NOT NULL,
    floor INTEGER NOT NULL CHECK (floor >= 0),
    status TEXT NOT NULL DEFAULT 'Available'
        CHECK (status IN ('Available','Occupied','Maintenance','Cleaning')),
    FOREIGN KEY (room_type_id) REFERENCES room_type(room_type_id)
);

```

Figure 2. Representative DDL Implementation in SQLite

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

1. Application Overview

The developed application provides a web-based interface for managing hotel reservations and guest services using the designed relational database. The application is developed using Python with the Flask framework and uses SQLite for database storage [5], [6]. It allows users to manage guest information, view room details, create reservations, manage stays, handle service requests and housekeeping activities, process invoices and payments, and view database-related information. The application communicates directly with the SQLite database so that the operations performed through the interface are stored and retrieved from the implemented database. Validation and database constraints are used to maintain data consistency and prevent invalid operations such as overlapping room reservations.

2. Development Environment and Database Connectivity

The application development environment tools used are:

Table 10. Application Development Environment

Item	Details
Programming Language	Python
DBMS	SQLite
IDE / Framework / GUI Toolkit	Flask
Database Connector / Driver	Python `sqlite3`
Development Environment	Visual Studio Code
Repository Link	https://github.com/Jasmitha144/hotel-reservation-dbms

3. Application Modules and Database Mapping

The application modules and database mapping are:

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations

Dashboard	Provides access to major application	Multiple tables	Read / Navigation

	functions		
Guest Management	Add and view guest information	GUEST	Create, Read
Room Management	View available and existing rooms	ROOM, ROOM_TYPE	Read / Search
Reservation Management	Create and manage room reservations	RESERVATION, GUEST, ROOM, RATE_PLAN	Create, Read
Stay Management	Manage guest check-in and stay information	STAY, RESERVATION	Create, Update
Service Management	Record guest service requests and charges	SERVICE_REQUEST, SERVICE_CHARGE, STAY	Create, Read
Housekeeping	Manage room housekeeping tasks	HOUSEKEEPING_TASK, ROOM	Create, Update
Billing / Invoice	Manage invoices and charges	INVOICE, STAY, SERVICE_CHARGE	Create, Read
Payment	Record payments against invoices	PAYMENT, INVOICE	Create, Read
Reports	Display database-related information	Multiple tables	Read / Query

4. CRUD Operations

The CRUD operations used are:

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Add Guest	GUEST	Guest record added successfully

Read / Search	View Guests / Rooms	GUEST, ROOM, ROOM_TYPE	Records displayed in application
Create	Create Reservation	RESERVATION	Reservation created for Room 201
Update	Stay / Reservation status	STAY, RESERVATION	Status updated during operation
Create	Record Payment	PAYMENT	Payment successfully recorded
Read	View Invoice / Payment details	INVOICE, PAYMENT	Transaction information displayed

5. Business Rules, Validation and Error Handling

The application uses input validation and database constraints to maintain data integrity. Required fields are validated before records are inserted into the database, while SQLite constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK further restrict invalid data. Reservation dates are validated so that the check-out date is later than the check-in date. The application also checks room availability before creating a reservation, preventing overlapping reservations for the same room. Invalid operations generate an error message instead of creating an incorrect database record. Database constraints help maintain data integrity by restricting invalid or inconsistent values [1], [4].

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Check-out must be after check-in	Database `CHECK` constraint	Invalid dates rejected
No overlapping reservation	Application availability validation	Overlapping Room 201 reservation rejected
Guest count must be positive	Database `CHECK` constraint	Invalid guest count rejected
Service charge cannot be negative	Database `CHECK` constraint	Negative charge rejected
Payment amount must be positive	Database `CHECK` constraint	Invalid payment rejected
Valid foreign-key references	SQLite foreign-key constraints	Invalid references rejected
Unique room number	`UNIQUE` constraint	Duplicate room numbers prevented

6. Reports and Database Operations

The application uses SQL database operations to retrieve and manage information stored in the relational database. Queries involving selection, filtering, joins, and aggregation can be used to obtain information such as room availability, guest records, reservations, service information, invoices, and payments. These operations demonstrate the use of the implemented relational database for practical hotel management tasks.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output

Room Availability	`SELECT`, filtering	Available rooms displayed
Guest Records	`SELECT`	Guest information displayed
Reservation Details	`SELECT`, joins	Reservation information displayed
Service Information	`SELECT`, joins	Service records displayed
Invoice Information	`SELECT`, joins	Billing information displayed

Payment Information	`SELECT`, joins	Payment records displayed

7. Application Screens

The developed application provides a user-friendly interface for performing the major hotel management operations. The following screenshots demonstrate the working application, including the dashboard, guest management, reservation processing, and payment transaction. These screens provide evidence that the application interacts with the implemented database.

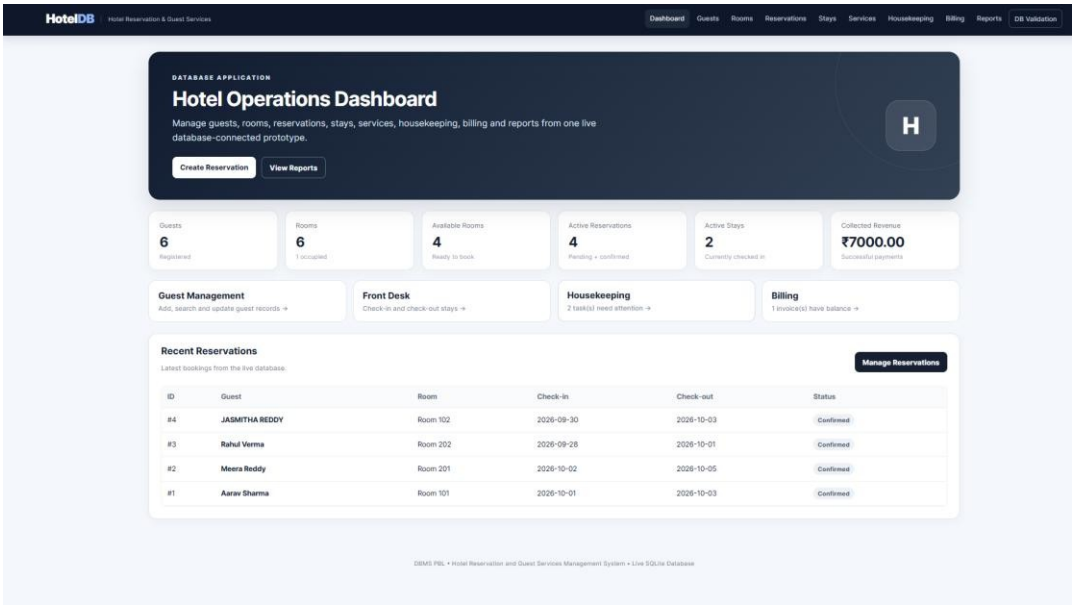


Figure 3. Hotel Reservation and Guest Services Management Dashboard

The dashboard provides access to the major modules of the hotel reservation and guest services management application.

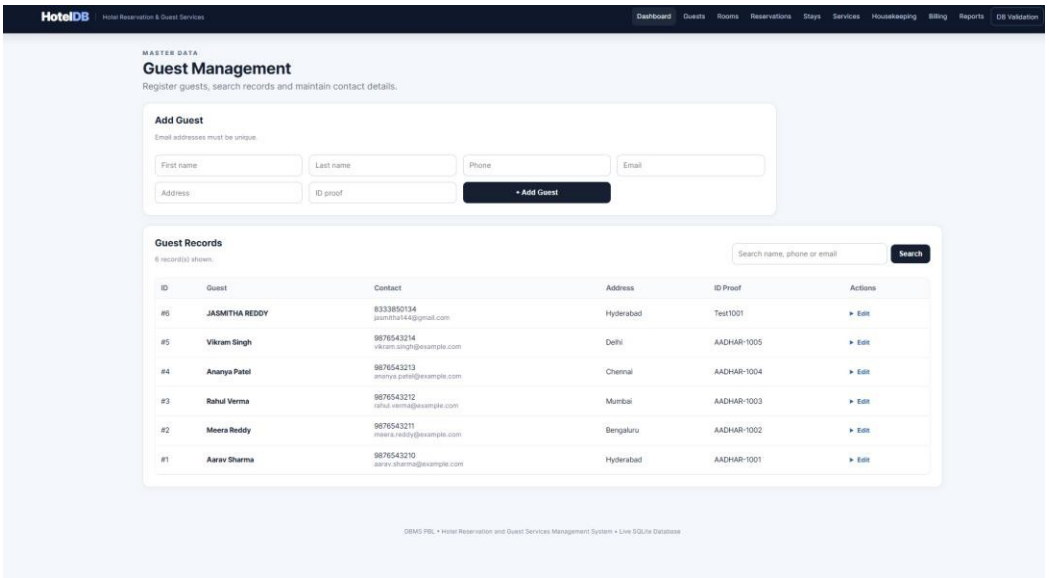


Figure 4. Guest Management and Guest Record Display

The Guest Management module allows guest information to be entered and retrieved from the database. The displayed record demonstrates successful storage and retrieval of guest details.

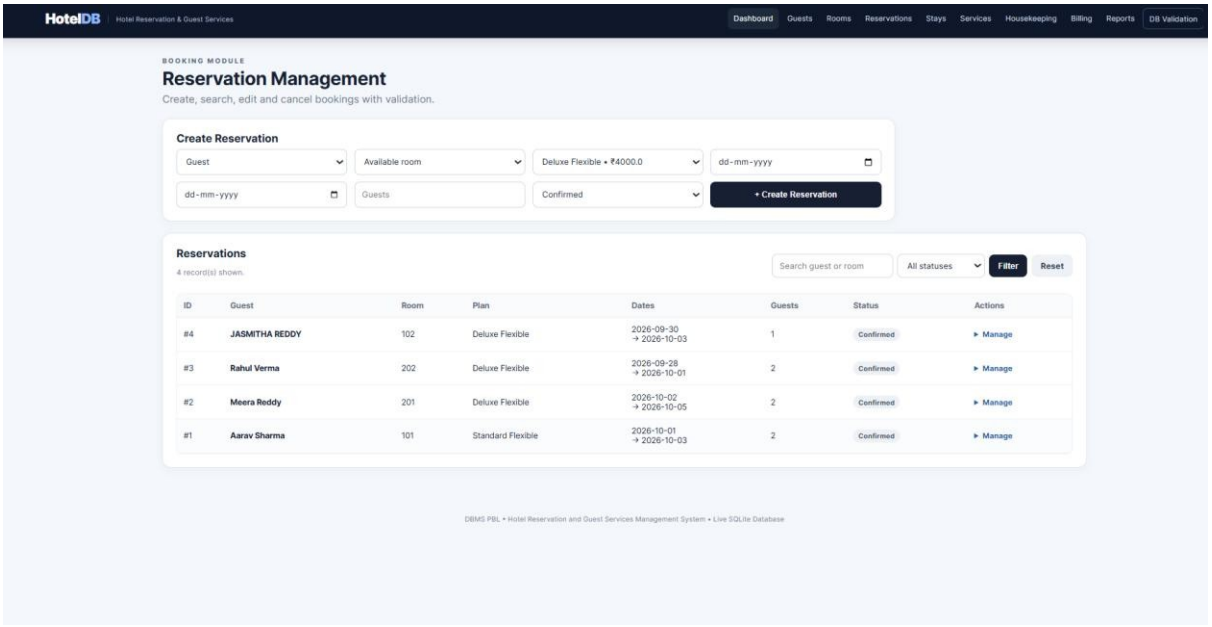


Figure 5. Room Reservation and Reservation Details

The Reservation module allows a guest to be associated with a room and rate plan for specified check-in and check-out dates. The displayed reservation demonstrates successful creation and retrieval of reservation information.

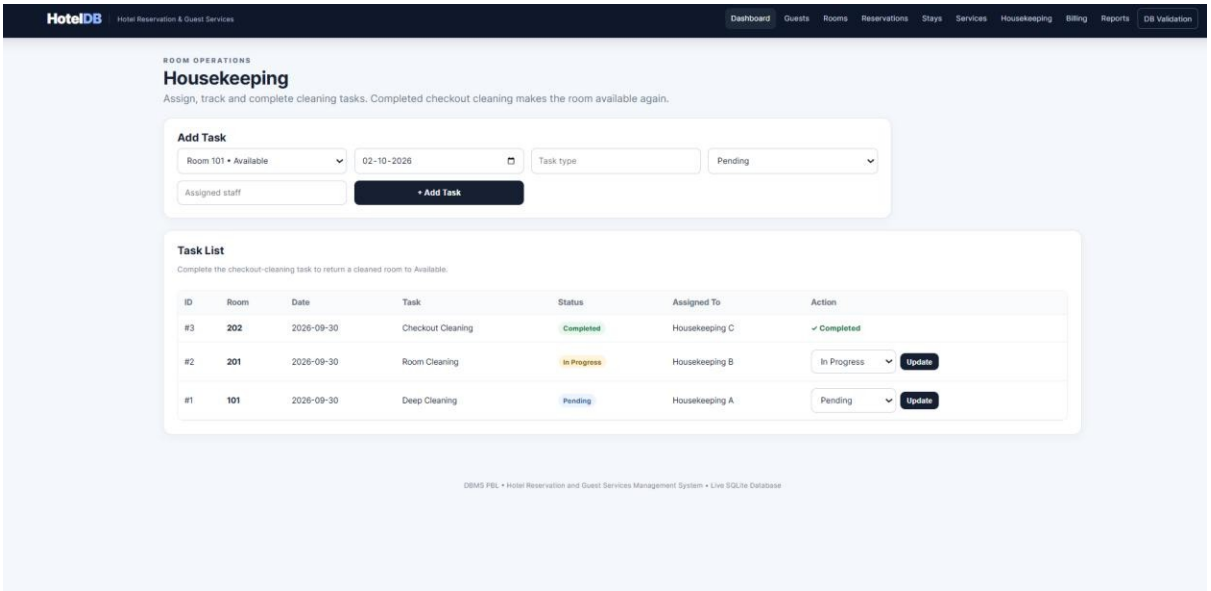


Figure 6. House Keeping Details

The housekeeping module allows hotel staff to manage housekeeping tasks associated with rooms. It helps track the task type, date, assigned staff, and task status for efficient room maintenance.

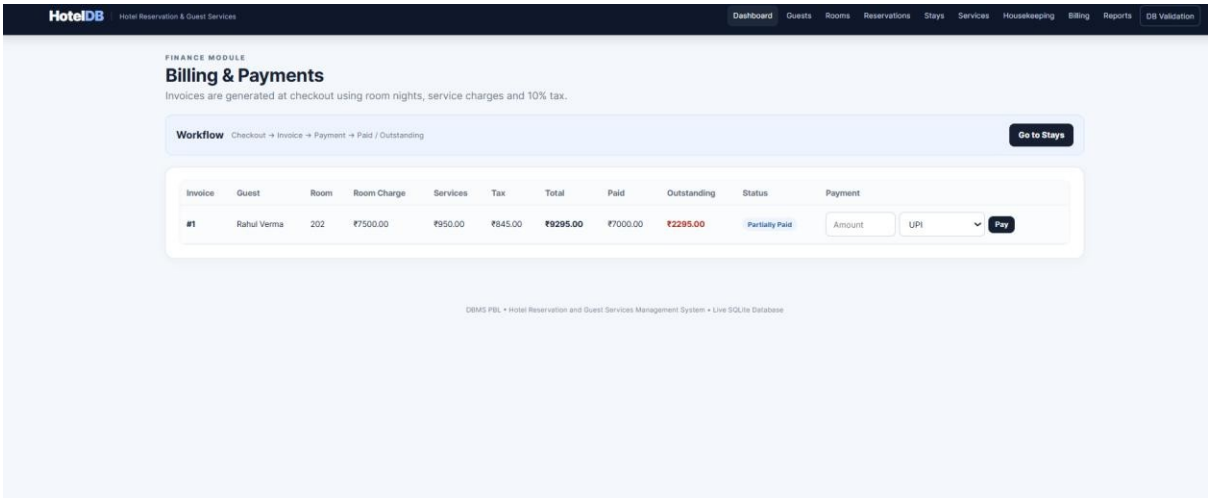


Figure 7. Payment Transaction and Payment Record

The Payment module records payment information against an invoice. The displayed transaction demonstrates successful storage of payment details in the database.

8. Testing and Results

The developed application was tested using positive and negative test cases to verify database operations, application functionality, and important business rules. The tests covered guest registration, room availability, reservation creation, overlapping reservation validation, and payment processing. The results were compared with the expected behaviour of the system.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Guest Registration	Add Jasmitha Reddy with valid guest details	Guest record should be created	Guest record was created successfully	Pass
TC-02	Room Availability	Check availability of Room 201	Available room should be displayed	Room 201 was displayed as available	Pass

TC-03	Reservation	Create reservation for Room 201 from 02-10-2026 to 04-10-2026	Reservation should be created	Reservation was created successfully	Pass
TC-	Overlapping	Try to reserve Room 201 from 03-10-	System should reject	Overlapping reservation was	Pass

04	Reservation	2026 to 05-10-2026	overlapping reservation	rejected	
TC -05	Payment	Record a valid payment against an invoice	Payment should be recorded	Payment was recorded successfully	Pass

CHAPTER 4
CONCLUSION

The Hotel Reservation and Guest Services Management System was successfully designed and implemented as a relational database application. The project demonstrates the practical application of DBMS concepts including Entity-Relationship modelling, relational schema design, functional dependencies, normalization up to Third Normal Form (3NF), primary and foreign keys, integrity constraints, CRUD operations, SQL queries, and database connectivity. The system provides modules for managing guests, rooms, reservations, stays, service requests, housekeeping tasks, invoices, and payments. The developed Flask application was connected to the SQLite database and tested using both valid and invalid operations. The testing results demonstrated successful data management and validation of important business rules such as preventing overlapping room reservations. Overall, the project demonstrates how a structured relational database can support the major operations involved in hotel reservation and guest services management.

1. Limitations and Future Enhancements

The current system is developed as a small-scale prototype using SQLite and is primarily designed to demonstrate database concepts and hotel management operations. The application has limited support for concurrent users and does not include integration with external hotel booking platforms or online payment gateways. Advanced features such as automated notifications, detailed analytics, and role-based access control are also not included in the current implementation.

In future, the system can be enhanced by migrating to a server-based DBMS such as MySQL or PostgreSQL to support larger-scale and multi-user hotel operations. Online payment gateway integration, email and SMS notifications, role-based authentication, advanced reports and analytics, and integration with external booking platforms can also be implemented. A mobile application could further improve accessibility for hotel staff and guests.

REFERENCES

1. A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

2. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

3. SQLite Documentation, "CREATE TABLE," SQLite, 2026. [Online]. Available: SQLite CREATE TABLE Documentation.

4. SQLite Documentation, "SQLite Foreign Key Support," SQLite, 2026. [Online]. Available: SQLite Foreign Key Documentation.

5. Python Software Foundation, "`sqlite3` — DB-API 2.0 interface for SQLite databases," *Python 3.14 Documentation*, 2026. [Online]. Available: Python sqlite3 Documentation.

6. Pallets Projects, "Flask Documentation," *Flask Documentation*, 2026. [Online]. Available: Flask Documentation.

7. Pallets Projects, "Quickstart," *Flask Documentation*, 2026. [Online]. Available: Flask Quickstart.

8. Oracle, "MySQL 8.0 Reference Manual," Oracle Corporation, 2026. [Online]. Available: MySQL Reference Manual.

9. PostgreSQL Global Development Group, "PostgreSQL Documentation," PostgreSQL, 2026. [Online]. Available: PostgreSQL Documentation.

10. A. Silberschatz, H. F. Korth, and S. Sudarshan, "Database Design and the Entity-Relationship Model," in *Database System Concepts*, 7th ed., McGraw-Hill, 2019.

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

Appendix A presents the complete SQL Data Definition Language (DDL) script used to create the hotel reservation and guest services management database. The script includes table definitions, primary keys, foreign keys, constraints, indexes, and other database-level rules implemented in SQLite.

database/schema.sql

APPENDIX B – SAMPLE DATA

Appendix B presents the sample data inserted into the database for testing and demonstration of the hotel reservation and guest services management system. The data includes room types, rooms, rate plans, guests, reservations, stays, service requests, housekeeping tasks, service charges, invoices, and payments.

database/seed.sql

APPENDIX C – SQL QUERIES

Appendix C presents the SQL queries used to retrieve and analyse information from the hotel database. The queries demonstrate operations such as data retrieval, filtering, joins, aggregation, sorting, and generation of useful hotel management information.

database/queries.sql

APPENDIX D – APPLICATION SOURCE COUDE

Appendix D contains the source code of the developed Python Flask application. The code demonstrates database connectivity, application routing, data insertion and retrieval, reservation processing, validation, housekeeping operations, billing, and payment processing.

app.py